

Laboratório — Algoritmos de ordenação

Disciplina: Estrutura de Dados — Aula 07

Professor: Alysson M. Bruno

Para fazer em casa, no seu ritmo, depois do encontro.

O que você vai fazer

1. **Ver** os três algoritmos andando passo a passo.
2. **Rodar** os três medindo o tempo de cada um.
3. **Conferir**, com o relógio, a previsão que a notação Big-O faz.
4. **Ordenar objetos** por mais de um critério.
5. **Escrever** o seu próprio Insertion Sort.

Tudo roda **no navegador**, inclusive no do celular. Nada é instalado.

Antes de começar

Tenha à mão:

- a apresentação e o texto-base desta aula (Google Classroom);
- os dois arquivos .java que você vai colar no JDoodle — eles vão **anexados a esta aula no Classroom** e estão também nos endereços abaixo, que abrem como texto puro:
 - OrdenacaoDemo.java — <https://raw.githubusercontent.com/alyssonbruno/publicacao/main/ED/AULA07/OrdenacaoDemo.java>
 - OrdenaProdutos.java — <https://raw.githubusercontent.com/alyssonbruno/publicacao/main/ED/AULA07/OrdenaProdutos.java>
- papel e caneta, ou o bloco de notas do celular, para **anotar os tempos** — sem anotar, o Passo 3 não funciona.

Quanto tempo leva? Os passos 1 a 3 são rápidos. O Passo 5 é o que dá trabalho e é o mais importante: reserve um momento em que você consiga pensar sem pressa.

Passo 1 — Ver o algoritmo andando

Abra o **visualizador** no navegador, neste endereço:

<https://alyssonbruno.github.io/publicacao/ED/AULA07/visualizador-ed-aula07.html>

Ele mostra **três coisas ao mesmo tempo**: a barra que muda de cor, a linha de código que está executando e o valor de cada variável naquele instante. Não pede conta nem instalação.

Repare que o passo a passo **não pula o laço**: o cabeçalho do for, o teste da condição e o `i++`, o `j++` e o `j--` são passos também. Logo abaixo das variáveis aparece a condição com os valores já substituídos — por exemplo `j`

$< n - 1 - i \rightarrow 0 < 4 \rightarrow \text{verdadeiro}$ —, que é o que explica por que o programa entrou no laço ou saiu dele.

No painel, a variável que **mudou** naquele passo fica destacada. A que aparece apagada, com um travessão, é a que ainda não existe ou já saiu do bloco em que foi declarada.

Faça, para **cada** um dos três algoritmos:

1. Execute com o vetor em ordem aleatória e acompanhe até o fim, **lendo o painel de variáveis** a cada passo.
2. Execute com o vetor **já ordenado**. Observe quantos passos cada um precisa.
3. Execute com o vetor em **ordem inversa**.

O que responder (anote):

- Qual dos três termina mais rápido com o vetor já ordenado? Por quê?
- O Selection Sort muda de comportamento quando o vetor já está ordenado?
- Quantas trocas o Selection Sort faz, comparado ao Bubble Sort?
- No Bubble Sort, em que passo a variável tmp deixa de aparecer apagada — e por que ela volta a ficar apagada logo depois?
- No Insertion Sort, durante os deslocamentos o vetor mostra um número **repetido**. De onde saiu a repetição, e onde está guardado o valor que sumiu?

Resultado esperado: o Insertion Sort termina quase de imediato com o vetor já ordenado — é o melhor caso $O(n)$ dele. O Selection Sort faz o mesmo número de comparações em qualquer entrada: ele não tem melhor caso.

A tmp só existe **dentro do if** que faz a troca: antes de entrar nele a variável nem foi declarada, e ao sair do bloco ela deixa de existir — por isso o painel a mostra apagada nos demais passos.

A repetição no Insertion Sort não é erro: ao deslocar, o programa **copia** $v[j]$ por cima de $v[j + 1]$, e por um instante o mesmo valor ocupa duas casas. O valor que estava ali foi salvo antes na variável chave — é justamente para isso que ela existe, e o painel mostra que ele não se perdeu.

Passo 2 — Rodar e medir

1. Abra <https://www.jdoodle.com/online-java-compiler-ide>.
2. Escolha a linguagem **Java** e a versão **JDK 25**.
3. Apague o exemplo que aparece na tela.
4. Cole o arquivo **OrdenacaoDemo.java** inteiro — pelo anexo do Classroom ou por <https://raw.githubusercontent.com/alyssonbruno/publicacao/main/ED/AULA07/OrdenacaoDemo.java>.
5. Clique em **Execute**.

O programa roda sozinho, sem pedir digitação.

Resultado esperado (os números variam de máquina para máquina):

n	Bubble (ms)	Selection (ms)	Insertion (ms)	Bubble/anterior
1000	0.85	0.31	0.09	-
2000	2.96	1.14	0.27	3.5x
4000	11.08	4.49	1.05	3.7x
8000	43.99	16.51	3.98	4.0x

Anote a sua tabela. Os seus números serão diferentes dos de cima, e tudo bem — o que importa é a **proporção** entre as linhas, não os milissegundos.

Passo 3 — Conferir a previsão do Big-O

Esta é a parte que dá sentido à aula. Com a sua tabela do Passo 2 na mão:

3.1 O tempo quadruplica?

De uma linha para a outra, **n dobra**. A teoria diz que, num algoritmo $O(n^2)$, o tempo deve ficar perto de **quatro vezes** maior.

Preencha:

De → para	Tempo antes	Tempo depois	Divida um pelo outro
1.000 → 2.000			
2.000 → 4.000			
4.000 → 8.000			

Resultado esperado: valores entre 3 e 5. A última linha às vezes passa de 4 — com oito mil elementos, o vetor já não cabe inteiro na memória rápida do processador e cada acesso passa a custar mais. Esse efeito não aparece na notação Big-O; só na medição.

3.2 Por que os três não empatam?

Os três algoritmos são $O(n^2)$. Mesmo assim, na sua tabela, o Bubble Sort gasta bem mais que o Insertion Sort.

Responda com as suas palavras: **por que a notação não distingue os dois?**

Resposta esperada: porque a notação descarta as constantes. Os dois crescem com n^2 , mas o Bubble Sort executa uma troca a cada comparação fora de ordem, enquanto o Insertion Sort apenas desloca. A diferença está na constante, e a constante é justamente o que a notação apaga. Big-O compara **crescimento**, não velocidade.

3.3 O experimento do aquecimento

No início do main existe um laço de aquecimento. Apague-o, rode de novo e compare a coluna Bubble/anterior com a de antes.

Resultado esperado: sem o aquecimento, a proporção deixa de ficar perto de 4x — costuma aparecer bem menor nas primeiras linhas. A máquina virtual do Java só traduz o código para linguagem de máquina depois de executá-lo algumas vezes; antes disso, ela interpreta, e as primeiras medições saem infladas. Lição de método: **medir desempenho exige preparar a medição.**

Passo 4 — Ordenar objetos

Cole o arquivo **OrdenaProdutos.java** no JDoodle e execute — pelo anexo do Classroom ou por <https://raw.githubusercontent.com/alyssonbruno/publicacao/main/ED/AULA07/OrdenaProdutos.java>.

Resultado esperado:

```
=== Por preço (crescente) ===
Mouse      R$   79.90
Teclado    R$  149.90
Webcam     R$  149.90
Headset    R$  249.90
Monitor    R$  899.00

=== Por estoque (decrescente) ===
Mouse      120 un.
Teclado    50 un.
Webcam     30 un.
Headset    15 un.
Monitor     8 un.
```

Agora faça a experiência:

1. Na declaração da lista, **troque Teclado e Webcam de lugar** e execute de novo. A ordem da primeira tabela muda?
2. Apague o `.thenComparing(Produto::nome)`, mantendo a troca do item 1, e execute. Agora muda?

Resultado esperado: com o `thenComparing`, a saída não muda — o critério de desempate é explícito e não depende da ordem de entrada. Sem ele, a saída passa a refletir a ordem em que os produtos foram declarados, porque o algoritmo interno do Java é **estável** e preserva a ordem original dos empates. É a diferença entre **garantir** o critério e **depende** da estabilidade.

Passo 5 — Escrever o seu

Implemente, à mão, o **Insertion Sort para a lista de produtos**, ordenando por preço, **sem** usar `Collections.sort` nem `.sort()`.

Exigências:

- receba um `Comparator<Produto>` como parâmetro, para o método servir a qualquer critério;

- não use nenhum método pronto de ordenação;
- imprima a lista antes e depois.

Esqueleto para começar:

```
static void insertionSort(List<Produto> lista, Comparator<Produto> criterio) {
    for (int i = 1; i < lista.size(); i++) {
        Produto chave = lista.get(i);
        int j = i - 1;
        // enquanto o elemento da esquerda for MAIOR que a chave, empurre-o para a
        // dica: criterio.compare(lista.get(j), chave) > 0
        // ...
    }
}
```

Teste com dois critérios diferentes — por preço e por nome — e confira se o mesmo método serve aos dois.

Passo 6 — Questões objetivas

Responda às questões no Google Classroom. Elas cobrem o conteúdo do encontro e o que você observou aqui.

Se você só tem o celular

Tudo acima funciona no navegador do celular. Duas dicas práticas:

- no JDoodle, use o modo paisagem: a tabela de resultados fica legível;
- para colar o arquivo, abra-o no Google Classroom, selecione todo o texto e cole.

Se ainda assim a execução não for possível, **leia** cada passo e as respostas esperadas em destaque: elas trazem o resultado e a explicação. O Passo 3, que é o mais importante, pode ser feito só com a tabela de exemplo deste roteiro e uma calculadora.

Como saber se você entendeu

Ao terminar, você deve conseguir responder, sem consultar:

1. Por que dobrar a entrada quadruplica o tempo de um algoritmo $O(n^2)$?
2. Por que dois algoritmos $O(n^2)$ podem ter desempenhos bem diferentes?
3. Em que situação o Insertion Sort chega a $O(n)$?
4. Por que o Selection Sort não tem melhor caso?
5. O que a estabilidade garante, e quando ela deixa de bastar?